

Token classification

প্রথম যে অ্যাপ্লিকেশনটি আমরা দেখবো তা হলো **token classification**।

এই কাজটি এমন সব সমস্যাকে বোঝায় যেখানে একটি sentence-এর প্রতিটি token (word)-এর জন্য একটি label নির্ধারণ করা হয়।

Token Classification এর উদাহরণ

1. Named Entity Recognition (NER)

একটি sentence-এর মধ্যে থাকা entity (যেমন: মানুষ, জায়গা, প্রতিষ্ঠান) খুঁজে বের করা।

এখানে প্রতিটি token-এর জন্য label দেওয়া হয়:

- entity হলে তার type
- না হলে “no entity”

2. Part-of-Speech Tagging (POS)

প্রতিটি শব্দের grammatical type নির্ধারণ করা:

- noun
- verb
- adjective ইত্যাদি

3. Chunking

একই entity-এর অংশ এমন tokenগুলোকে group করা।

এখানে label হয়:

- B- → chunk-এর শুরু
- I- → chunk-এর ভিতরে
- O → কোনো chunk-এর অংশ না

এগুলো ছাড়াও আরও অনেক ধরনের token classification problem আছে।

এখানে আমরা **BERT model** ব্যবহার করে **NER task fine-tune** করবো।

Preparing the data

প্রথমে আমাদের একটি dataset লাগবে।

এখানে আমরা **CoNLL-2003 dataset** ব্যবহার করবো, যা Reuters-এর news data দিয়ে তৈরি।

Important Note

যদি তোমার dataset এভাবে থাকে:

- text → word list
- label → প্রতিটি word-এর label

তাহলে তুমি এই method নিজের dataset-এও ব্যবহার করতে পারবে।

Load Dataset

Code

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset("conll2003")
```

ব্যাখ্যা

- এটি dataset download করবে
- cache-এ save করবে

Dataset Structure

Code

```
raw_datasets
```

Output

```
DatasetDict({
  train: Dataset({
    features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
    num_rows: 14041
  })
  validation: Dataset({
    features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
```

```
num_rows: 3250
})
test: Dataset({
  features: ['chunk_tags', 'id', 'ner_tags', 'pos_tags', 'tokens'],
  num_rows: 3453
})
})
```

ব্যাখ্যা

- Dataset তিন ভাগে ভাগ করা:
 - train
 - validation
 - test
- প্রতিটিতে ৫টি column আছে:
 - tokens → শব্দগুলোর list
 - ner_tags → NER label
 - pos_tags → POS label
 - chunk_tags → chunk label
 - id → sample id

Important Difference

এখানে text sentence আকারে নেই
বরং **word list (tokens)** আকারে আছে

Example Data দেখার জন্য

Code

```
raw_datasets["train"][0]["tokens"]
```

Output

```
['EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'lamb', '.']
```

ব্যাখ্যা

এটি একটি sentence, কিন্তু word list আকারে দেওয়া হয়েছে

NER Labels দেখার জন্য

Code

```
raw_datasets["train"][0]["ner_tags"]
```

Output

```
[3, 0, 7, 0, 0, 0, 7, 0, 0]
```

ব্যাখ্যা

- এগুলো integer label
- model training-এর জন্য ব্যবহার করা হয়
- কিন্তু সরাসরি বুঝা কঠিন

Label Names বের করা

Code

```
ner_feature = raw_datasets["train"].features["ner_tags"]  
ner_feature
```

Output

```
Sequence(feature=ClassLabel(num_classes=9, names=['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG',  
'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC'], names_file=None, id=None), length=-1, id=None)
```

Label Names List

Code

```
label_names = ner_feature.feature.names  
label_names
```

Output

['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC']

Label Meaning

- O → কোনো entity না
- B-PER / I-PER → person entity
- B-ORG / I-ORG → organization
- B-LOC / I-LOC → location
- B-MISC / I-MISC → অন্যান্য entity

Labels Decode করা

Code

```
words = raw_datasets["train"][0]["tokens"]
labels = raw_datasets["train"][0]["ner_tags"]

line1 = ""
line2 = ""

for word, label in zip(words, labels):
    full_label = label_names[label]
    max_length = max(len(word), len(full_label))
    line1 += word + " " * (max_length - len(word) + 1)
    line2 += full_label + " " * (max_length - len(full_label) + 1)

print(line1)
print(line2)
```

Output

```
'EU   rejects German call to boycott British lamb .'
'B-ORG O    B-MISC O    O O    B-MISC O    O'
```

ব্যাখ্যা

এখানে দেখা যাচ্ছে:

- EU → B-ORG (organization)

- German → B-MISC
- British → B-MISC

অর্থাৎ কিছু শব্দ entity হিসেবে label করা হয়েছে

B- এবং I- Label Example

Code

'Germany \\'s representative to the European Union \\'s veterinary committee Werner Zwingmann said on Wednesday consumers should buy sheepmeat from countries other than Britain until the scientific advice was clearer .'

'B-LOC O O O O B-ORG I-ORG O O O B-PER I-PER O O O
O O O O O O O O B-LOC O O O O O O O O'

ব্যাখ্যা

এখানে গুরুত্বপূর্ণ বিষয়:

- **European Union**
 - European → B-ORG
 - Union → I-ORG
- **Werner Zwingmann**
 - Werner → B-PER
 - Zwingmann → I-PER

অর্থাৎ:

- B- → entity-এর শুরু
- I- → entity-এর ভিতরের অংশ

Processing the data

আগের মতোই, আমাদের text-গুলোকে model বুঝতে পারার জন্য **token IDs**-এ **convert** করতে হবে। কিন্তু token classification-এর ক্ষেত্রে একটি বড় পার্থক্য হলো:

- এখানে input আগে থেকেই **word** আকারে **split** করা (**pre-tokenized**) থাকে

ভাগ্য ভালো যে tokenizer API এটি সহজেই handle করতে পারে।

আমাদের শুধু একটি special flag ব্যবহার করতে হবে।

Tokenizer তৈরি করা

Code

```
from transformers import AutoTokenizer
```

```
model_checkpoint = "bert-base-cased"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

ব্যাখ্যা

- আমরা এখানে BERT-এর pretrained tokenizer ব্যবহার করছি
- এটি Hugging Face Hub থেকে download হবে

Fast Tokenizer চেক করা

Code

```
tokenizer.is_fast
```

Output

True

ব্যাখ্যা

- True মানে এটি **fast tokenizer**
- fast tokenizer হলে আমরা extra সুবিধা পাই (যেমন word_ids)

Pre-tokenized Input Tokenize করা

Code

```
inputs = tokenizer(raw_datasets["train"][0]["tokens"], is_split_into_words=True)
inputs.tokens()
```

Output

```
[[CLS], 'EU', 'rejects', 'German', 'call', 'to', 'boycott', 'British', 'la', '##mb', '.', [SEP]]
```

ব্যাখ্যা

- [CLS] → sentence শুরু
- [SEP] → sentence শেষ
- বেশিরভাগ word একই থাকে
- কিন্তু "lamb" → la + ##mb (subword এ split হয়েছে)

Problem: Label Mismatch

- original labels = 9 টি
- tokenized input = 12 টি

কারণ:

- special tokens যোগ হয়েছে
- কিছু word subword-এ split হয়েছে

তাই আমাদের labels adjust করতে হবে

Token to Word Mapping

Code

```
inputs.word_ids()
```

Output

```
[None, 0, 1, 2, 3, 4, 5, 6, 7, 7, 8, None]
```

ব্যাখ্যা

- প্রতিটি token কোন word থেকে এসেছে তা দেখায়
- **None** → special token
- **7, 7** → একই word (lamb → 2 token)

Label Align করার Function

Code

```
def align_labels_with_tokens(labels, word_ids):  
    new_labels = []  
    current_word = None  
    for word_id in word_ids:  
        if word_id != current_word:  
            # Start of a new word!  
            current_word = word_id  
            label = -100 if word_id is None else labels[word_id]  
            new_labels.append(label)
```

```

elif word_id is None:

    # Special token

    new_labels.append(-100)

else:

    # Same word as previous token

    label = labels[word_id]

    # If the label is B-XXX we change it to I-XXX

    if label % 2 == 1:

        label += 1

    new_labels.append(label)

return new_labels

```

সহজ ব্যাখ্যা

এই function যা করে:

1. Special token → -100 দেয়
2. একই word-এর সব token → একই label
3. যদি B- label হয় → ভিতরের token-এ I- বানায়

Function Test করা

Code

```

labels = raw_datasets["train"][0]["ner_tags"]

word_ids = inputs.word_ids()

print(labels)

```

```
print(align_labels_with_tokens(labels, word_ids))
```

Output

```
[3, 0, 7, 0, 0, 0, 7, 0, 0]
```

```
[-100, 3, 0, 7, 0, 0, 0, 7, 0, 0, 0, -100]
```

ব্যাখ্যা

- শুরু ও শেষে **-100** (special token)
- **"lamb"** split হওয়ায় extra label যোগ হয়েছে
- model training-এ **-100** ignore করা হবে

Alternative Idea

কিছু researcher শুধু:

- word-এর প্রথম token-এ label দেয়
- বাকিগুলোতে **-100** দেয়

এতে:

- long word বেশি impact ফেলতে পারে না

Full Dataset Preprocessing

আমাদের এখন পুরো dataset process করতে হবে:

- tokenize করতে হবে
- labels align করতে হবে

Function তৈরি

```
def tokenize_and_align_labels(examples):
```

```
    tokenized_inputs = tokenizer(
```

```

        examples["tokens"], truncation=True, is_split_into_words=True
    )
    all_labels = examples["ner_tags"]
    new_labels = []
    for i, labels in enumerate(all_labels):
        word_ids = tokenized_inputs.word_ids(i)
        new_labels.append(aligned_labels_with_tokens(labels, word_ids))

    tokenized_inputs["labels"] = new_labels
    return tokenized_inputs

```

ব্যাখ্যা

- একসাথে অনেক data tokenize করা হয়
- প্রতিটি example-এর জন্য label align করা হয়
- শেষে labels add করা হয়

Dataset-এ Apply করা

Code

```

tokenized_datasets = raw_datasets.map(
    tokenize_and_align_labels,
    batched=True,
    remove_columns=raw_datasets["train"].column_names,
)

```

ব্যাখ্যা

- পুরো dataset process করা হয়
- `batched=True` → দ্রুত কাজ করে
- পুরোনো column remove করে

Fine-tuning the model with the Trainer API

Trainer ব্যবহার করে training করার code আগের মতোই থাকবে।

শুধু দুইটি জায়গায় পরিবর্তন আছে:

- data batch করার পদ্ধতি (data collation)
- metric calculation function

Data collation

আমরা Chapter 3-এর মতো `DataCollatorWithPadding` ব্যবহার করতে পারি না, কারণ:

- এটি শুধু input (input_ids, attention_mask, token_type_ids) pad করে
- কিন্তু এখানে labels-ও একইভাবে pad করতে হবে

এবং padding-এর সময় labels-এর জন্য `-100` ব্যবহার করতে হবে, যাতে loss calculation-এ এগুলো ignore হয়।

DataCollatorForTokenClassification ব্যবহার

Code

```
from transformers import DataCollatorForTokenClassification
```

```
data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
```

ব্যাখ্যা

- এটি input এবং label দুইটাই একসাথে pad করে
- label padding-এ `-100` ব্যবহার করে

Test করা

Code

```
batch = data_collator([tokenized_datasets["train"][i] for i in range(2)])
```

```
batch["labels"]
```

Output

```
tensor([[ -100,   3,   0,   7,   0,   0,   0,   7,   0,   0,   0,  -100],
        [ -100,   1,   2, -100, -100, -100, -100, -100, -100, -100, -100, -100]])
```

ব্যাখ্যা

- দুইটি sample একসাথে batch হয়েছে
- ছোট label list-টি বড়টার সমান করতে **-100** দিয়ে pad করা হয়েছে

Compare করা

Code

```
for i in range(2):  
    print(tokenized_datasets["train"][i]["labels"])
```

Output

```
[-100, 3, 0, 7, 0, 0, 0, 7, 0, 0, 0, -100]  
[-100, 1, 2, -100]
```

ব্যাখ্যা

- দ্বিতীয় sample ছোট ছিল
- তাই batch-এ গিয়ে **-100** দিয়ে বড় করা হয়েছে

Metrics

Trainer-এ প্রতি epoch শেষে metric বের করতে হলে `compute_metrics()` function লাগবে।

sequeval install করা

Code

```
!pip install sequeval
```

Metric load করা

Code

```
import evaluate
```

```
metric = evaluate.load("sequeval")
```

Label decode করা

Code

```
labels = raw_datasets["train"][0]["ner_tags"]
```

```
labels = [label_names[i] for i in labels]
```

```
labels
```

Output

```
['B-ORG', 'O', 'B-MISC', 'O', 'O', 'O', 'B-MISC', 'O', 'O']
```

ব্যাখ্যা

- integer label → string label-এ convert করা হয়েছে

Fake prediction তৈরি

Code

```
predictions = labels.copy()
```

```
predictions[2] = "O"
```

```
metric.compute(predictions=[predictions], references=[labels])
```

Output

```
{'MISC': {'precision': 1.0, 'recall': 0.5, 'f1': 0.67, 'number': 2},  
'ORG': {'precision': 1.0, 'recall': 1.0, 'f1': 1.0, 'number': 1},  
'overall_precision': 1.0,  
'overall_recall': 0.67,  
'overall_f1': 0.8,  
'overall_accuracy': 0.89}
```

ব্যাখ্যা

- প্রতিটি entity-এর জন্য:
 - precision
 - recall
 - F1 score
- overall score-ও দেখায়

compute_metrics function

Code

```
import numpy as np
```

```
def compute_metrics(eval_preds):
```

```
    logits, labels = eval_preds
```

```
    predictions = np.argmax(logits, axis=-1)
```

```
    # Remove ignored index (special tokens) and convert to labels
```

```
    true_labels = [[label_names[l] for l in label if l != -100] for label in labels]
```

```
    true_predictions = [
```

```
        [label_names[p] for (p, l) in zip(prediction, label) if l != -100]
```

```
        for prediction, label in zip(predictions, labels)
```

```
    ]
```

```
    all_metrics = metric.compute(predictions=true_predictions, references=true_labels)
```

```
    return {
```

```
        "precision": all_metrics["overall_precision"],
```

```
        "recall": all_metrics["overall_recall"],
```

```
        "f1": all_metrics["overall_f1"],
```

```
        "accuracy": all_metrics["overall_accuracy"],
```

```
    }
```

ব্যাখ্যা

এই function যা করে:

1. logits $\rightarrow$ prediction বের করে (**argmax**)

2. **-100** label remove করে
3. integer → string label convert করে
4. metric calculate করে

Defining the model

Token classification-এর জন্য আমরা ব্যবহার করবো:

Code

```
id2label = {i: label for i, label in enumerate(label_names)}
```

```
label2id = {v: k for k, v in id2label.items()}
```

Model load করা

```
from transformers import AutoModelForTokenClassification
```

```
model = AutoModelForTokenClassification.from_pretrained(  
    model_checkpoint,  
    id2label=id2label,  
    label2id=label2id,  
)
```

ব্যাখ্যা

- pretrained BERT model load করা হয়েছে
- নতুন classification head add হয়েছে

Label সংখ্যা চেক করা

```
model.config.num_labels
```

Output

9

ব্যাখ্যা

- dataset-এ মোট 9টি label আছে
- model-এর সাথে match করছে

Fine-tuning শুরু করার আগে

Hugging Face login

```
from huggingface_hub import notebook_login
```

```
notebook_login()
```

Training Arguments

Code

```
from transformers import TrainingArguments
```

```
args = TrainingArguments(
```

```
    "bert-finetuned-ner",
```

```
evaluation_strategy="epoch",  
save_strategy="epoch",  
learning_rate=2e-5,  
num_train_epochs=3,  
weight_decay=0.01,  
push_to_hub=True,  
)
```

ব্যাখ্যা

- learning_rate → learning speed
- epochs → কয়বার train হবে
- weight_decay → regularization
- push_to_hub → model upload করবে

Trainer তৈরি করা

Code

```
from transformers import Trainer
```

```
trainer = Trainer(  
    model=model,  
    args=args,  
    train_dataset=tokenized_datasets["train"],  
    eval_dataset=tokenized_datasets["validation"],  
    data_collator=data_collator,  
    compute_metrics=compute_metrics,
```

```
processing_class=tokenizer,  
)
```

Training শুরু

```
trainer.train()
```

ব্যাখ্যা

- model train শুরু হবে
- প্রতি epoch শেষে evaluate করবে
- Hub-এ upload হবে

Model Upload

```
trainer.push_to_hub(commit_message="Training complete")
```

Output

```
'https://huggingface.co/sgugger/bert-finetuned-ner/commit/26ab21e5b1568f9afeccdaed2d8715f571d786ed'
```

ব্যাখ্যা

- model Hub-এ upload হয়েছে
- link থেকে check করা যাবে

Final Result

- Trainer নিজে থেকেই model card তৈরি করে
- evaluation result save করে

- inference widget দিয়ে test করা যায়

A custom training loop

এখন আমরা পুরো **training loop** দেখবো, যাতে তুমি নিজের মতো করে সব কিছু customize করতে পারো। এটি Chapter 3-এর মতোই হবে, তবে evaluation অংশে কিছু পরিবর্তন আছে।

Preparing everything for training

প্রথমে আমাদের dataset থেকে **DataLoader** তৈরি করতে হবে।

DataLoader তৈরি

Code

```
from torch.utils.data import DataLoader
```

```
train_dataloader = DataLoader(
```

```
    tokenized_datasets["train"],
```

```
    shuffle=True,
```

```
    collate_fn=data_collator,
```

```
    batch_size=8,
```

```
)
```

```
eval_dataloader = DataLoader(
```

```
    tokenized_datasets["validation"], collate_fn=data_collator, batch_size=8
```

```
)
```

ব্যাখ্যা

- train_dataloader → training data (shuffle করা হয়েছে)
- eval_dataloader → validation data (shuffle করা হয়নি)
- batch_size=8 → একবারে ৮টি sample

Model আবার load করা

Code

```
model = AutoModelForTokenClassification.from_pretrained(  
    model_checkpoint,  
    id2label=id2label,  
    label2id=label2id,  
)
```

ব্যাখ্যা

- নতুন করে model load করা হয়েছে
- আগের training থেকে continue না করে fresh শুরু

Optimizer

Code

```
from torch.optim import AdamW  
  
optimizer = AdamW(model.parameters(), lr=2e-5)
```

ব্যাখ্যা

- AdamW optimizer ব্যবহার করা হয়েছে
- learning rate = $2e-5$

Accelerator ব্যবহার

Code

```
from accelerate import Accelerator

accelerator = Accelerator()

model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

ব্যাখ্যা

- GPU/TPU ব্যবহারের জন্য prepare করা হয়
- distributed training সহজ করে

Learning Rate Scheduler

Code

```
from transformers import get_scheduler

num_train_epochs = 3

num_update_steps_per_epoch = len(train_dataloader)

num_training_steps = num_train_epochs * num_update_steps_per_epoch

lr_scheduler = get_scheduler(
    "linear",
```

```
optimizer=optimizer,  
num_warmup_steps=0,  
num_training_steps=num_training_steps,  
)
```

ব্যাখ্যা

- learning rate ধীরে ধীরে কমানো হয়
- total training steps হিসাব করা হয়েছে

Hugging Face Repository তৈরি

Code

```
from huggingface_hub import Repository, get_full_repo_name
```

```
model_name = "bert-finetuned-ner-accelerate"  
repo_name = get_full_repo_name(model_name)  
repo_name
```

Output

```
'sgugger/bert-finetuned-ner-accelerate'
```

ব্যাখ্যা

- model upload করার জন্য repo তৈরি করা হচ্ছে

Repository clone করা

```
output_dir = "bert-finetuned-ner-accelerate"

repo = Repository(output_dir, clone_from=repo_name)
```

Training loop

Postprocess function

```
def postprocess(predictions, labels):

    predictions = predictions.detach().cpu().clone().numpy()

    labels = labels.detach().cpu().clone().numpy()

    # Remove ignored index (special tokens) and convert to labels

    true_labels = [[label_names[l] for l in label if l != -100] for label in labels]

    true_predictions = [

        [label_names[p] for (p, l) in zip(prediction, label) if l != -100]

        for prediction, label in zip(predictions, labels)

    ]

    return true_labels, true_predictions
```

ব্যাখ্যা

- tensor → numpy করা হয়
- -100 বাদ দেওয়া হয়
- integer → string label করা হয়

Full Training Loop

Code

```
from tqdm.auto import tqdm

import torch

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):

    # Training

    model.train()

    for batch in train_dataloader:

        outputs = model(**batch)

        loss = outputs.loss

        accelerator.backward(loss)

    optimizer.step()

    lr_scheduler.step()

    optimizer.zero_grad()

    progress_bar.update(1)

    # Evaluation

    model.eval()

    for batch in eval_dataloader:

        with torch.no_grad():

            outputs = model(**batch)
```

```

predictions = outputs.logits.argmax(dim=-1)

labels = batch["labels"]

# Necessary to pad predictions and labels for being gathered
predictions = accelerator.pad_across_processes(predictions, dim=1, pad_index=-100)
labels = accelerator.pad_across_processes(labels, dim=1, pad_index=-100)

predictions_gathered = accelerator.gather(predictions)
labels_gathered = accelerator.gather(labels)

true_predictions, true_labels = postprocess(predictions_gathered, labels_gathered)

metric.add_batch(predictions=true_predictions, references=true_labels)

results = metric.compute()

print(
    f"epoch {epoch}:",
    {
        key: results[f"overall_{key}"]
        for key in ["precision", "recall", "f1", "accuracy"]
    },
)

# Save and upload

```

```
accelerator.wait_for_everyone()

unwrapped_model = accelerator.unwrap_model(model)

unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)

if accelerator.is_main_process:

    tokenizer.save_pretrained(output_dir)

    repo.push_to_hub(

        commit_message=f"Training in progress epoch {epoch}", blocking=False

    )
```

ব্যখ্যা (সহজভাবে)

এই loop তিনটি অংশে কাজ করে:

1. Training

- model train mode-এ যায়
- forward pass → loss বের হয়
- backward → gradient update

2. Evaluation

- model eval mode-এ যায়
- prediction বের করে
- padding ঠিক করা হয়
- metric calculate করা হয়

3. Save & Upload

- model save করা হয়
- Hugging Face Hub-এ upload করা হয়

Model Save করার Code ব্যখ্যা

```
accelerator.wait_for_everyone()
```

```
unwrapped_model = accelerator.unwrap_model(model)
```

```
unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
```

ব্যাখ্যা

- সব process wait করে
- model unwrap করা হয়
- তারপর save করা হয়

Fine-tuned Model ব্যবহার

Code

```
from transformers import pipeline
```

```
# Replace this with your own checkpoint
```

```
model_checkpoint = "huggingface-course/bert-finetuned-ner"
```

```
token_classifier = pipeline(
```

```
    "token-classification", model=model_checkpoint, aggregation_strategy="simple"
```

```
)
```

```
token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

Output

```
[{'entity_group': 'PER', 'score': 0.9988506, 'word': 'Sylvain', 'start': 11, 'end': 18},
```

```
{'entity_group': 'ORG', 'score': 0.9647625, 'word': 'Hugging Face', 'start': 33, 'end': 45},
```

```
{'entity_group': 'LOC', 'score': 0.9986118, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```


ব্যাখ্যা

- Sylvain → Person
- Hugging Face → Organization
- Brooklyn → Location